Lab 3 - Written Justification

System: Restaurant Table Booking & Pre-Ordering App
Scenario: Problem Statement #36

1. Scenario review

The system lets diners view table availability, choose a table on an interactive floor plan, reserve a date and time, and place food pre-orders. Restaurant managers maintain reservations and table status, while the kitchen receives order details and preparation information.

2. Architectural style analysis

Layered architecture separates presentation, business logic and data management. It is easy to understand and maintain, although requests may pass through several layers and individual functions are less straightforward to scale independently.

Microservices architecture divides the system into independently deployable services. It supports independent scaling and fault isolation, but adds operational complexity, network latency and data-consistency challenges.

Client-server architecture centralizes processing and data on a server used by multiple clients. It simplifies centralized control and data management, but may create a server bottleneck and a single point of failure.

3. Architecture selection

We chose Layered Architecture for the Restaurant Table Booking & Pre-Ordering App.

4. Reasons for the selection

1. Clear separation of responsibilities: diner and manager screens remain in the presentation layer, reservation and order rules stay in the business layer, and persistent records are handled by the data layer. This makes the application easier to test and maintain.

2. Consistent reservation processing: table selection, availability checks and order updates can use common business rules and data access. This reduces duplicated logic and helps keep reservation information consistent across user interfaces.

5. Security advantage

Authentication and role-based authorization can restrict diner and manager actions. Encrypting communications with HTTPS/TLS helps protect personal and reservation information in transit.

6. Performance benefit

Separating availability checks from order processing allows each operation to use focused queries and business logic. Efficient database access and caching can help support the required table-status synchronization target of under 500 ms; this target must still be verified by testing.